SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment - 2

Name of Student : K.Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task 1: Cleaning Sensor Data

❖ Scenario:You are cleaning IoT sensor data where negative values are invalid.

❖ Task:Use Gemini in Colab to generate a function that filters out all negative numbers from a list. ❖ Expected Output:

➢ Before/after list

➢ Screenshot of Colab execution

Prompt:-

I have a list of IoT sensor readings, but some values are negative and those are invalid. Write a simple Python function that removes all negative numbers and returns only the valid (zero or positive) readings. The original list should not be changed.

Code:-

```
Assignment2.py > ...
#I have a list of IoT sensor readings, but some values are negative and those are inval
def remove_negative_readings(readings):
    return [reading for reading in readings if reading >= 0]
# Example usage:
original_readings = [23, -5, 0, 15, -2, 42]
valid_readings = remove_negative_readings(original_readings)
print("Original readings:", original_readings)  # Output: [23, -5, 0, 15, -2, 42]
print("Valid readings:", valid_readings)        # Output: [23, 0, 15, 42]
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users
Original readings: [23, -5, 0, 15, -2, 42]
Valid readings: [23, 0, 15, 42]
```

Justification:-

I used this approach because sensor data should not be modified directly. The function creates a new list that contains only valid values, which keeps the

original data safe. It is simple, readable, and easy to understand even for beginners.

Task 2: String Character Analysis

❖ Scenario:

You are building a text-analysis feature.

❖ Task:

Use Gemini to generate a Python function that counts vowels, consonants, and digits in a string. ❖ Expected Output:

➢ Working function

➢ Sample inputs and outputs

Prompt:-

Write a Python function that takes a sentence as input and counts how many vowels, consonants, and digits are present in it. It should work the same for both uppercase and lowercase letters and return the counts clearly.

Code:-

```
#task 2
#Write a Python function that takes a sentence as input and counts how many
def count_characters(sentence):
    vowels = "aeiouAEIOU"
    consonants = "bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ"
    digits = "0123456789"

    vowel_count = sum(1 for char in sentence if char in vowels)
    consonant_count = sum(1 for char in sentence if char in consonants)
    digit_count = sum(1 for char in sentence if char in digits)

    return {
        'vowels': vowel_count,
        'consonants': consonant_count,
        'digits': digit_count
    }
# Example usage:
input_sentence = "Hello World! 123"
counts = count_characters(input_sentence)
print(f'In the sentence "{input_sentence}":')
print(f'Vowels: {counts["vowels"]}')
print(f'Consonants: {counts["consonants"]}')
print(f'Digits: {counts["digits"]}')
```

Output:-

```
In the sentence "Hello World! 123":
Vowels: 3
Consonants: 7
Digits: 3
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:- This function checks each character one by one and identifies whether it is a vowel, consonant, or digit. Using built-in string methods makes the logic clean and easy to follow. It helps in understanding how character classification works in Python.

Task 3: Palindrome Check – Tool Comparison ❖

Scenario:

You must decide which AI tool is clearer for string logic.

❖ Task:

Generate a palindrome-checking function using Gemini and Copilot, then compare the results.

❖ Expected Output:

➢ Side-by-side code comparison

➢ Observations on clarity and structure

Prompt:-

Create a Python function that checks whether a given string is a palindrome. It should ignore spaces, special characters, and uppercase or lowercase differences. The function should return True or False

Code:-

```python
#Task 3
#Create a Python function that checks whether a given string is a palindrome. It sho
def is_palindrome(text):
    cleaned_text = ''.join(char for char in text if char.isalnum()).lower()
    return cleaned_text == cleaned_text[::-1]
# Example usage:
input_string = "A human, a man, a camel, banana"
result = is_palindrome(input_string)
print(f'Is the string "{input_string}" a palindrome? {result}')  # Output: True
```

Output:-

```
Is the string "A human, a man, a camel, banana" a palindrome? False
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:-

The function first cleans the string to remove unnecessary characters and converts everything to lowercase. After that, it compares the string with its reverse. This step-by-step method makes the logic simple and easy to explain to beginners.

Task 4: Code Explanation Using AI ❖ Scenario:

You are reviewing unfamiliar code written by another developer.

❖ Task:

Ask Gemini to explain a Python function (prime check OR palindrome check) line by line. ❖ Expected Output:

➢ Code snippet

➢ AI explanation

➢ Student comments on understanding

Prompt:-

Provide a Python function to check whether a number is prime (or whether a string is a palindrome) and explain each line of the code in simple words so that a first-year student can understand it clearly.

Code:-

```
#Task 4
#Provide a Python function to check whether a number is prime (or whether a string
def is_prime(n):
    # Check if the number is less than or equal to 1, which are not prime numbers
    if n <= 1:
        return False
    # Loop from 2 to the square root of n (inclusive)
    for i in range(2, int(n**0.5) + 1):
        # If n is divisible by any number in this range, it is not prime
        if n % i == 0:
            return False
    # If n is not divisible by any number in the range, it is prime
    return True
# Example usage:
number = input("Enter a number: ")
try:
    number = int(number)
    result = is_prime(number)
    print(f'Is the number {number} prime? {result}')
except ValueError:
    print("Invalid input. Please enter a valid integer.")
```

Output:-

```
Enter a number: 12
Is the number 12 prime? False
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:-

The explanation is written in a step-by-step manner so that each line of the code becomes clear. It breaks down the logic into small parts, which helps in better understanding and reduces confusion for beginners.